

Ejercicio 1 – Detectando el entorno

Abre PowerShell y ejecuta:

```
$PSVersionTable
```

1. → Comprueba la versión de PowerShell (Si Core 7+ → es multiplataforma).

Luego abre Git Bash o WSL y ejecuta (deberás tener instalado un wsl...):

```
echo $0
```

```
uname -a
```

2. → Identifica la shell y la versión del kernel.

Preguntas:

● ¿Qué diferencias observas entre la información que devuelve PowerShell y Bash?

PowerShell muestra los datos de forma que se entiende mejor en una tabla que podemos ver que dato es de que, pero en Bash te lo escupe todo en una línea.

● ¿Cuál crees que se ajusta mejor al estándar POSIX? (No te estoy pidiendo que lo sepas, solo que

Bash porque cumple el estándar POSIX perfectamente ya que podrás ejecutar sus instrucciones en otros sistemas operativos.

Ejercicio 2 – Comparando comandos básicos

Acción	PowerShell	Bash (POSIX)
Mostrar directorio actual	Get-Location	pwd
Listar archivos	Get-ChildItem	ls -l
Cambiar carpeta	Set-Location	cd carpeta
Borrar archivo	Remove-Item archivo.txt	rm archivo.txt
Buscar texto en archivo	Select-String "texto" archivo.txt	grep "texto" archivo.txt
Ver procesos	Get-Process	ps -ef

Tarea:

● Ejecuta cada comando en ambos entornos.

● Anota qué comando te parece más “natural” y cuál ofrece más información.

Get-Location me parece más natural porque el comando se entiende que te va a dar perfectamente que te va a dar la ubicación y te la muestra hasta casi la raíz.

Get-ChildItem se entiende mejor que salida te va a dar, pero hay que decir que ls -l muestra más información.

Set-Location es más directo y se entiende por el nombre lo que hace.

Remove-Item me parece más natural.

Select-String es más natural y detallado.

Get-Process es más natural y visible.

En resumen, todos los comandos de PowerShell son mas natural porque los nombres se entienden mejor para saber la salida, pero en algunas salidas son mejor de Bash y además tengo que añadir que pueden ser mejor para trabajar más rápido ya que son nombres mas cortos.

● Discute en grupo si PowerShell se puede considerar “compatible” a nivel conceptual con POSIX.

Juanjo, Fran y Yo hemos discutido la pregunta y hemos llegado a la conclusión que no lo cumple porque creemos que las instrucciones deben poder ejecutarse en diferentes sistemas.

Ejercicio 3 – Script paralelo

Crea un script equivalente en ambos entornos que muestre información básica del sistema y del

usuario.

Versión bash (POSIX)

```
#!/bin/sh
```

```
echo "=== INFORME DEL SISTEMA ==="
```

```
echo "Usuario: $(whoami)"
```

```
echo "Sistema operativo: $(uname -o)"
```

```
echo "Procesos activos: $(ps | wc -l)"
```

```
echo "Directorio actual: $(pwd)"
```

Versión PowerShell

```
# informe_sistema.ps1
```

```
Write-Host "=== INFORME DEL SISTEMA ==="
```

```
Write-Host "Usuario: $env:USERNAME"
```

```
Write-Host "Sistema operativo: $env:OS"
```

```
Write-Host "Procesos activos:" (Get-Process).Count
```

```
Write-Host "Directorio actual:" (Get-Location)
```

Tareas:

1. Ejecútalos y guarda el resultado en un archivo de texto (> salida.txt).

2. Observa las diferencias en la estructura y salida.

En PowerShell muestra la ruta de forma más completa.

3. Reflexiona: ¿cuál cumple con las normas POSIX y por qué?

Bash cumple con las normas porque esta basado en los estándares de POSIX.

Ejercicio 4 – Compatibilidad de scripts

1. Copia el script POSIX (informe_sistema.sh) a tu entorno PowerShell o CMD.

Intenta ejecutarlo:

```
bash informe_sistema.sh
```

2. (O con WSL si está instalado.)

3. Si falla, analiza por qué.

Ahora prueba lo contrario: ejecuta un script .ps1 desde bash usando:

```
pwsh ./informe_sistema.ps1
```

Preguntas:

- ¿Qué barreras encuentras para la compatibilidad total entre PowerShell y POSIX?
Que nunca serán totalmente compatibles por la sintaxis.
- ¿Qué soluciones o herramientas crees que lo facilitan?

WSL es una solución ya que consigues poder ejecutar los archivos .sh dentro de Windows.